

ANN and DL (CAI-361)

Objectives:

- **Error/Loss Function**
- **Chain Rule for Backpropagation**

Error/Cost/Loss function in Neural Network

The objective of the training process is to find the weights (W) and biases (b) that minimize the error. This minimization is achieved through the **gradient descent** algorithm. To begin with, the weights are randomly initialized, and an iterative process of a subtle weights change is performed until convergence.

Each iteration begins with a forward pass, that outputs the current prediction and evaluates the model's error by a cost function/Loss function/error function. Next, a backward pass is conducted to compute the weights gradient. To find the best parameters that minimize the error, we use the gradient descent algorithm. The goal is to find the minimum point of the cost function, where the gradient is close to zero. The algorithm iteratively moves, step by step, in the direction of steepest descent, toward the minimum point. The step size is named the '**learning rate**', and it is a scalar that determines how much the weights change in each iteration, and therefore how quickly the NN converges. The learning rate is a hyperparameter that has to be tuned. Smaller learning rates require more training time, whereas larger learning rates result in rapid training, but might suffer from compromised performances and instability.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{where } \hat{y}_i = w^t x_i + b$$

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (w^t x_i + b))^2$$

$$\frac{\partial L}{\partial w} = \Delta w = \frac{1}{n} \sum_{i=1}^n -2 x_i (y_i - (w^t x_i + b))$$

$$\frac{\partial L}{\partial b} = \Delta b = \frac{1}{n} \sum_{i=1}^n -2 (y_i - (w^t x_i + b))$$

To update the weights, the gradients are multiplied by the learning rate (alpha), and the new weights are calculated by the following formula.

$$w_{\text{new}} = w_{\text{old}} - \eta \Delta w$$

$$b_{\text{new}} = b_{\text{old}} - \eta \Delta b$$

As the model iterates, the gradient gradually converges toward zero, where the error is most likely the lowest (Alternatively, the model might converge to a local optimum and present sub-optimal performances). But this weight updation is only applicable when there is only input and output layers in the data.

When we have one or more hidden layers in a Neural Network then to update weights/biases of whole network is carried out using Chain Rule.

Updating weights of Multilayer NN using Chain Rule

This article will follow the structure of a two layered Neural Network, where X (also named A[0]) is the input vector, A[1] is a hidden layer, and Y-hat is the output layer. The basic architecture of the net is illustrated in the picture below (The neurons amount in each layer are irrelevant to the equations, since we are using the vectorized equations):

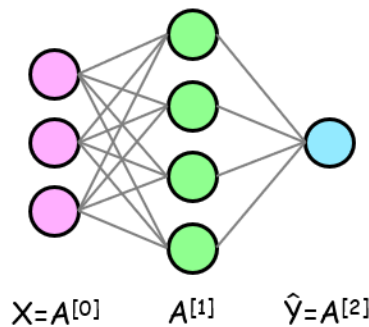


Figure1. Two layered neural network

Forward pass

To perceive how the backward propagation is calculated, we first need to overview the forward propagation. Our net starts with a vectorized linear equation, where the layer number is indicated in square brackets.

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

Equation 1. Straight line equation.

Next, a non linear **activation function** (A) is added. This activation enables the net to break linearity and adjust to complex patterns in the data. Several different activation functions can be used (e.g. sigmoid, ReLU, tanh), and here we will use the sigmoid activation for our NN.

$$A^{[1]} = \sigma(Z^{[1]}) = \frac{1}{1 + e^{-z}}$$

Equation 2. Sigmoid activation.

So far we computed the first layer. The second layer, like the first, is composed of a linear equation ($Z^{[2]}$), followed by a sigmoid activation ($A^{[2]}$). Since this is the last layer in our net, the activation result ($A^{[2]}$) is the model's prediction (Y-hat).

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]}) = \frac{1}{1 + e^{-z}}$$

Equations 3,4 . Straight line and sigmoid equations respectively.

Finally, to evaluate and minimize the error, we define a **cost function**. Here we are using the ‘Mean Squared Error’ (MSE) function. For simplicity, we will use Stochastic Gradient Descent (SGD) method, meaning that only one sample is processed in each iteration.

$$\text{Cost } (J) = \frac{1}{2} (\hat{Y} - Y)^2$$

Equation 5. Mean Squared Error cost function.

We can summarize the computational graph of the forward pass

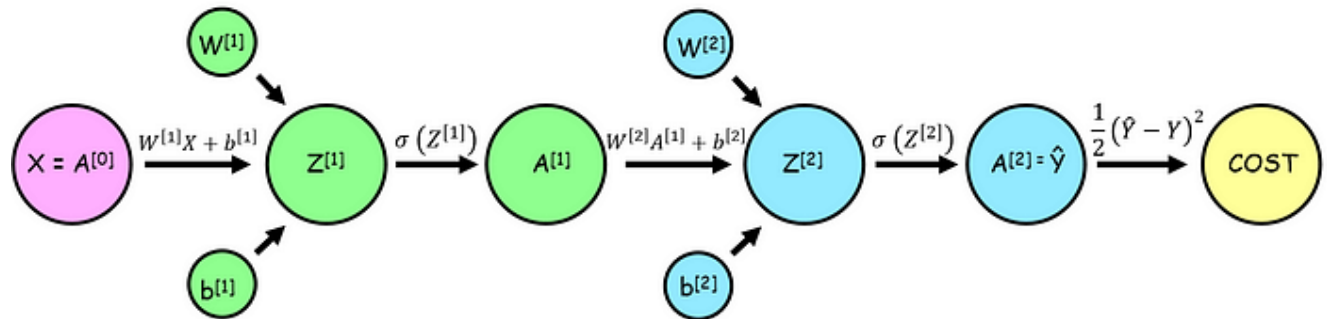


Figure 5. Computational graph for the forward pass

Backward pass

In order to update the weights and biases after each iteration, we need to compute the gradients. In our two layered net there are 4 parameters to be updated: $W[2]$, $b[2]$, $W[1]$ and $b[1]$, and therefore 4 gradients to be computed:

$$\frac{\partial J}{\partial W[2]}, \quad \frac{\partial J}{\partial b[2]}, \quad \frac{\partial J}{\partial W[1]}, \quad \frac{\partial J}{\partial b[1]}$$

But how are we going to find the derivatives of these composite functions? According to the ‘**chain rule**’, we’ll construct the product of the derivatives along all paths connecting the variables. Let’s follow the weights of the second layer ($W[2]$) gradient:

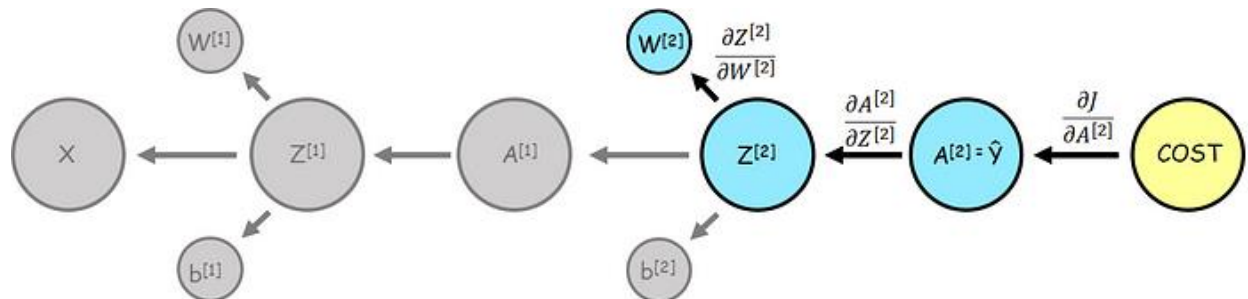


Figure3. $W[2]$ gradient

From the diagram above we can clearly see that the change in the cost J with respect to $W[2]$ is:

$$\frac{\partial J}{\partial W[2]} = \frac{\partial J}{\partial A[2]} \frac{\partial A[2]}{\partial Z[2]} \frac{\partial Z[2]}{\partial W[2]}$$

Equation 6. Gradient of the cost J with respect to the weights of layer two $W[2]$

To resolve this, we'll start by computing the partial derivative of the cost J with respect to $A[2]$, which is also the prediction $\hat{Y}$. The original cost function is shown on the left, and the derivative on the right:

Original function:

$$J = \frac{1}{2} (A^{[2]} - Y)^2$$

Partial derivative:

$$\left| \frac{\partial J}{\partial A^{[2]}} = A^{[2]} - Y \right.$$

Equation 7. MSE cost function (left) and its partial derivative with respect to $A[2]$ (right).

The partial derivative of the sigmoid activation in $A[2]$, with respect to $Z[2]$ is represented by the following]):

Original function:

$$A^{[2]} = \sigma(Z^{[2]}) = \frac{1}{1 + e^{-Z^{[2]}}}$$

Partial derivative:

$$\left| \frac{\partial A^{[2]}}{\partial Z^{[2]}} = A^{[2]}(1 - A^{[2]}) \right.$$

Equation 8. Sigmoid activation on the second layer (left) and it's partial derivative with respect to $Z[2]$ (right).

The partial derivative of $Z[2]$ with respect to the weights $W[2]$:

Original function:

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

Partial derivative:

$$\left| \frac{\partial Z^{[2]}}{\partial W^{[2]}} = A^{[1]} \right.$$

Equation 9. Straight line equation on the second layer (left), and it's partial derivative with respect to $W[2]$ (right).

Lets chain everything together, to compute the $W[2]$ gradient:

$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W^{[2]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot A^{[1]}$$

Equation 10. Gradient of the cost J with respect to the weights of layer two ($W[2]$)

Next, we will compute the $b[2]$ gradient in a similar way. Lets follow the gradient:

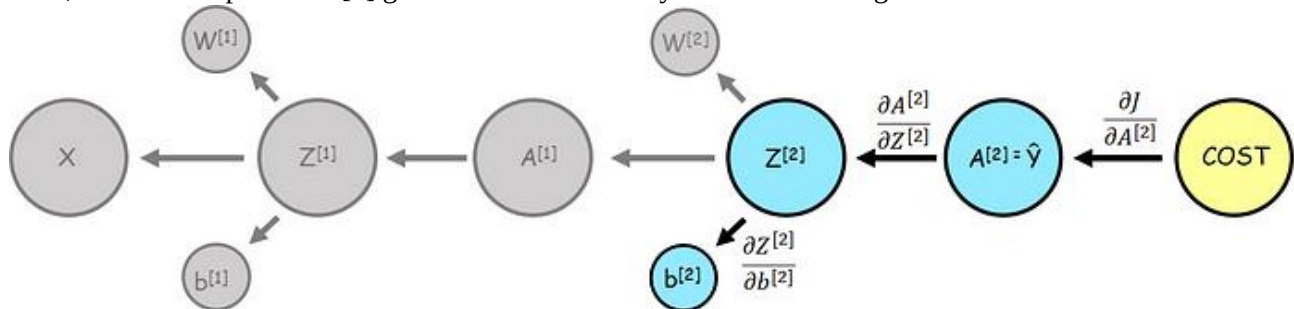


Figure 4. Gradient of the cost J with respect to the bias of layer two $b[2]$

$$\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

Equation 11. Gradient of the cost J with respect to the bias of layer two (b[2])

The first two parts of the b[2] gradient were already calculated above (The partial derivative of the cost J with respect to Z[2]), and the last part is equal to 1:

Original function:

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

Partial derivative:

$$\frac{\partial Z^{[2]}}{\partial b^{[2]}} = 1$$

Equation 12. Straight line equation for the second layer (left), and it's partial derivative with respect to b[2](right).

So the overall b[2] gradient is:

$$\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial b^{[2]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot 1$$

Equation 13. The gradient of the cost J with respect to the weights of layer two W[2]

At last, we finished computing the gradients for the second layer. The gradients for first layer are a bit longer, but we already computed parts of it. Lets follow the gradient for W[1] update:

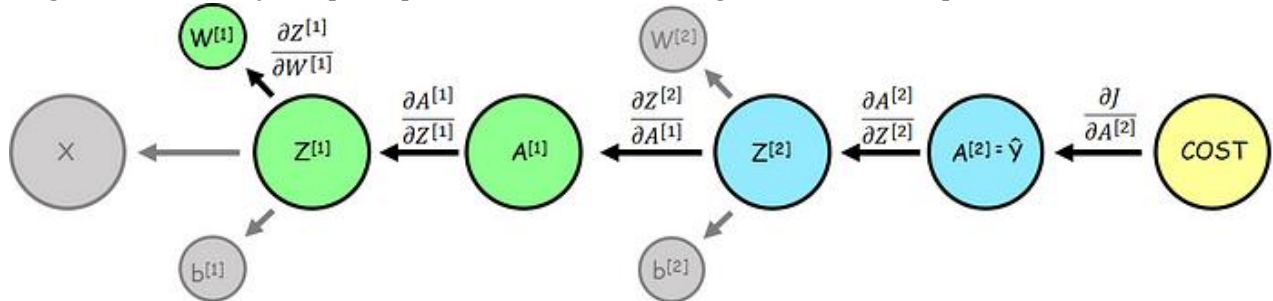


Figure 5. W[1] gradient

The first two parts of the gradient were previously calculated for layer 2. The partial derivative of Z[2] with respect to A[1], is W[2]:

Original function:

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

Partial derivative:

$$\frac{\partial Z^{[2]}}{\partial A^{[1]}} = W^{[2]}$$

Equation 14. Straight line equation for the second layer (left), and it's partial derivative with respect to A[1](right).

The last two parts are calculated in the same manner as it was calculated in layer 2. Taking it together, we get:

$$\frac{\partial J}{\partial W^{[1]}} = \underbrace{\frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}}}_{\text{Previously calculated}} \frac{\partial Z^{[1]}}{\partial W^{[1]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot W^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \cdot X$$

Equation 15. The gradient of the cost J with respect to the weights of the first layer W[1]

And if we follow the b[1] gradient:

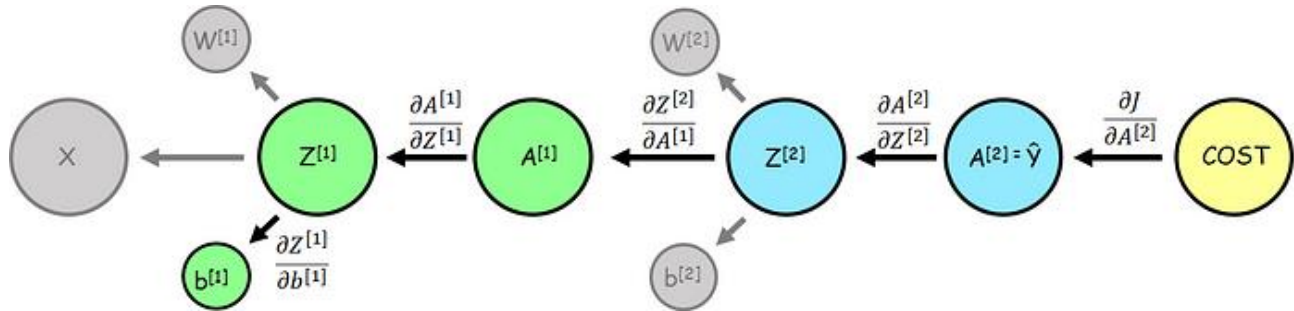


Figure 6. b[1] gradient

We'll get:

$$\frac{\partial J}{\partial b^{[1]}} = \underbrace{\frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}}}_{\text{Previously calculated}} \frac{\partial Z^{[1]}}{\partial b^{[1]}} = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]}) \cdot W^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \cdot 1$$

Equation 16. The gradient of the cost J with respect to the bias of the first layer b[1]

By this part we finished computing all the gradients of the weights and biases for one iteration of our NN.

Weight update

Once computing the gradients, we can update the new parameters for the model, and iterate all over again until the model converges. Denote that alpha is the learning rate, a hyperparameter that will determine the convergence rate.

$$\begin{aligned} W_{new}^{[1]} &= W_{old}^{[1]} - \alpha \frac{dJ}{dW^{[1]}} & | & & b_{new}^{[1]} &= b_{old}^{[1]} - \alpha \frac{dJ}{db^{[1]}} \\ W_{new}^{[2]} &= W_{old}^{[2]} - \alpha \frac{dJ}{dW^{[2]}} & | & & b_{new}^{[2]} &= b_{old}^{[2]} - \alpha \frac{dJ}{db^{[2]}} \end{aligned}$$

Weights update. W = Weights, alpha = Learning rate, J = Cost. Layer number is denoted in square brackets.

Home Task 1:

Consider the following NN. Assume hidden and output neurons are using Sigmoid function. All biases are zero. Actual output is 0.5 and the learning rate is 1. Perform Forward and Backward pass on the network. Findout the changes of weights and biases after forward pass.

